VisualLogic

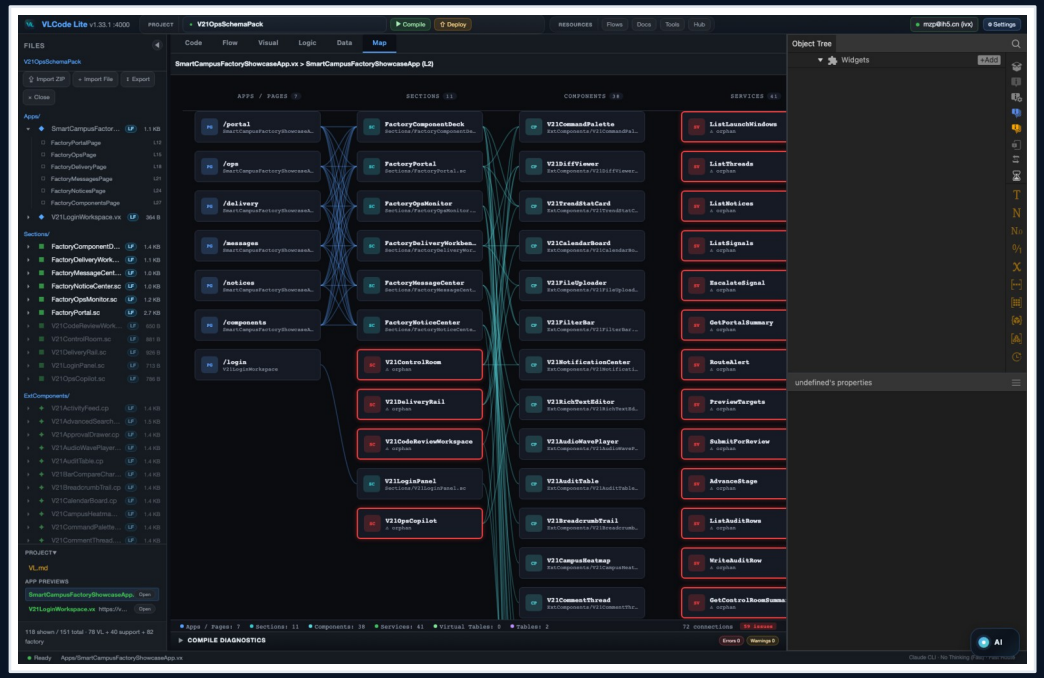
LOCAL · AI-NATIVE · ENTERPRISE APP BASE

Tame AI inside a DAG.

Ship complete, deployable enterprise applications — backend and all, not toys.

Open source (MIT) · Local-first

github.com/VisualLogic-AI/VL-Code



Map mode — a whole VL app as one architecture graph.

01 THE FRAME

Not AI-assisted. AI-native.

VL Language

Component-oriented,
minimal but complete

VL IDE

A graphical projection
of the language

AI Generation

AI-first,
multi-agent parallel

Not Low-Code · Not No-Code · Not DSL — general-purpose, no capability ceiling.

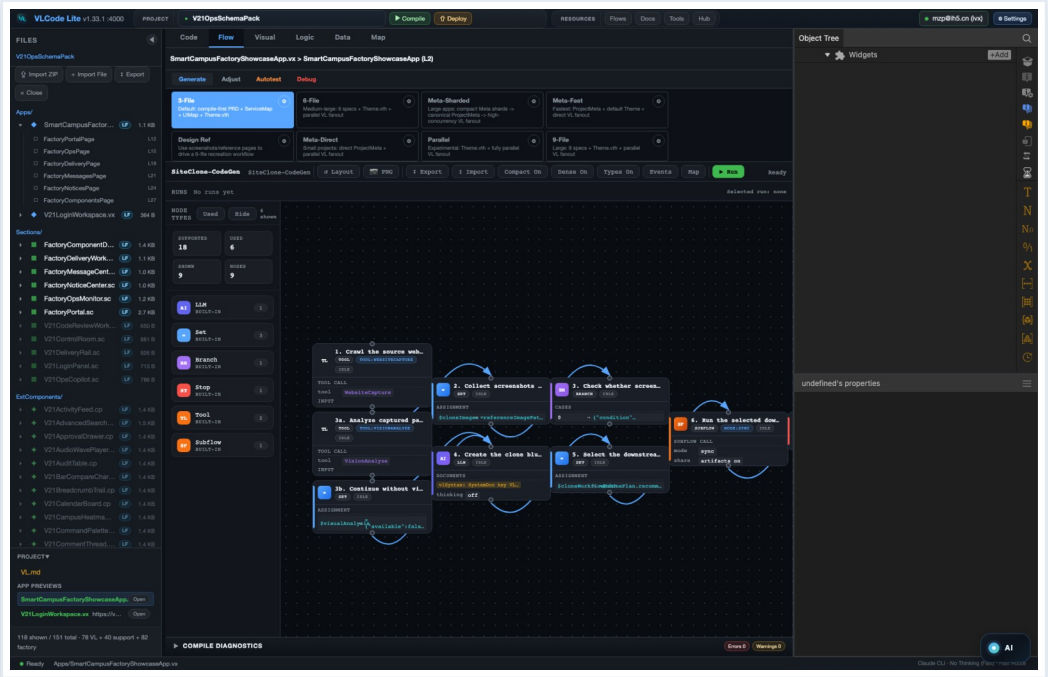
> *Low-code **hides** complexity. VL **structures** it.*

02 THE SPINE

AI is brilliant at a point. It can't hold a line.

So VLCode puts the AI inside a DAG.

Neural nets, approval flows, dependency chains, data pipelines — most real program logic is nodes plus directed edges. Put AI in that graph: creativity lives inside each node, determinism lives in the edges. You stop babysitting and start orchestrating.



Flow mode — each build runs as a verifiable DAG.

> ***Same AI. A 10× gap between people. A DAG is what flattens it.***

03 THE MOAT

VL is AI's Intermediate Representation.

Representation efficiency > model size

Human intent → VL (IR) → executable React / Node. Whoever controls the representation layer controls the system's ceiling.

One ~100 KB Bible. Infinite apps.

The whole VL syntax fits in ~100 KB — UI, logic, backend, database, deployment. Traditional full-stack means learning 50+ framework docs. VL needs one source of truth.

04 THE HARNESS

A game needs a prompt. An enterprise app needs a harness.

A game lives inside the prompt — rules, assets, state, feedback are self-contained. A real enterprise app cannot: it must connect backend resources, data contracts, identities, permissions, approvals, deployment, logs, and human decisions across many steps. That long, grounded process is AI coding's dead angle — not the code, the harness.

VLCode = the harness, productized. Freeze repeatable flows, resource patterns, validation habits and team knowledge into a product — so you orchestrate instead of carrying it all in your head.

05 VLCODE CAPABILITIES · I

Enterprise-grade, end to end.

Enterprise AI coding

An agentic orchestrator drives ~34 tools across the whole build loop — not a side chat box. Pluggable providers (Anthropic, OpenAI, DeepSeek, Kimi, GLM, Gemini). Feature Safety Guard preflights every change with rollback checkpoints.

Complete applications

VL covers frontend + backend + data + theme in one language. Deployed apps read real database rows — not mockups.

Strong backend

Typed ServiceDomain methods, VirtualTable projections with vector search, typed Field contracts, service-level auth. The backend is first-class, not an afterthought.

06 VLCODE CAPABILITIES · II

Fast, reusable, governable.

Fast & parallel generation

Concurrent waves up to concurrency 24. Grounded: 150 methods + 30 pages, 0 lint errors, 476s; a faster run: 108 methods + 90 pages, 233s.

The Component Factory

Install real .wc components with install contracts. Every install dual-writes to the Factory and the Resource Center — build once, reuse and share instantly.

Powerful workflow DAG

VL Workflow Engine (spec 4.1): checkpoint, resume, rerun-from-step, breakpoints, human Review gates, Actor execution, and Score / Grow / Channel / Topology / Iteration.

Customize & share everything

Components, flows, skills, docs, themes, prompts — all customizable, shared through the Resource Center's review-gated lifecycle (local → cloud-private → submitted → platform-shared).

07 ENTERPRISE BACKEND, BUILT-IN

Cloud, identity, permissions — as components.

The harness that real enterprise apps need — assembled, not bolted on afterward.

Backend · cloud integration

Typed ServiceDomain methods + VirtualTable projections with vector search bind real backend & cloud capabilities — first-class, not glue.

User components

Identity & user-management building blocks (UserAuth + user tables), plus your own .cp components — reusable through the Component Factory.

Permission components

Service-level auth declares authn / authz semantics; data-auth components and platform permission config gate every access.

08 THE NUMBERS

Verifiable, not hopeful.

÷10	×10	10×
tokens	speed	scale

1 line of VL ≈ 200–400 lines of code · ~80% less code · ~90% token savings · 500+ components beyond context limits.

> ***From “looks right” to verified. From probability to process.***

Every DAG node can run lint / compile / preview / test / human gate — runs are reproducible and auditable.

09 LOCAL-FIRST

Local-first. Cloud optional. Export = zero lock-in.

Fully offline core

Lint / Compile / codegen run against a pinned local parser — no cookie, no project ID, no network.

Export anywhere

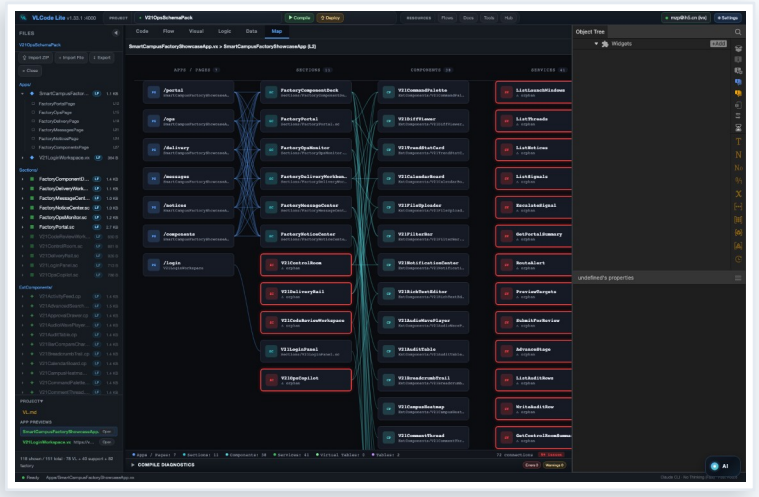
Export as portable .vl, .vl + dev state, a VL .zip, or a .vflow. Or export React + Node source and self-host.

Cloud deploy is an option

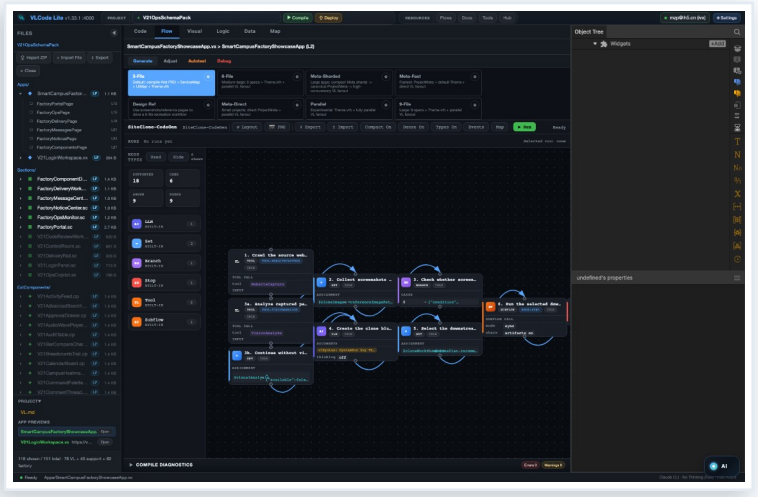
Production publish with its own production DB + real-domain URL is there when you want it — an option, not a cage.

You reason in pictures, not buried text.

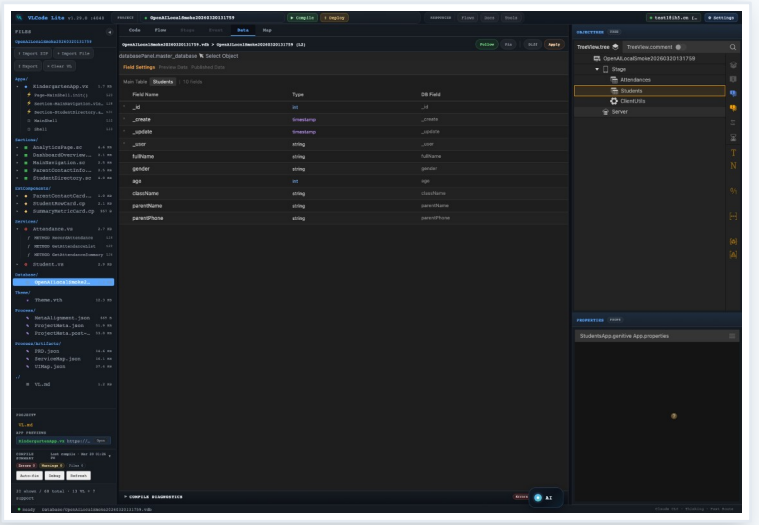
Architecture → Map · Process → DAG + Event · Data → ER. Code ↔ Graph, 1:1 bidirectional.



Map — architecture



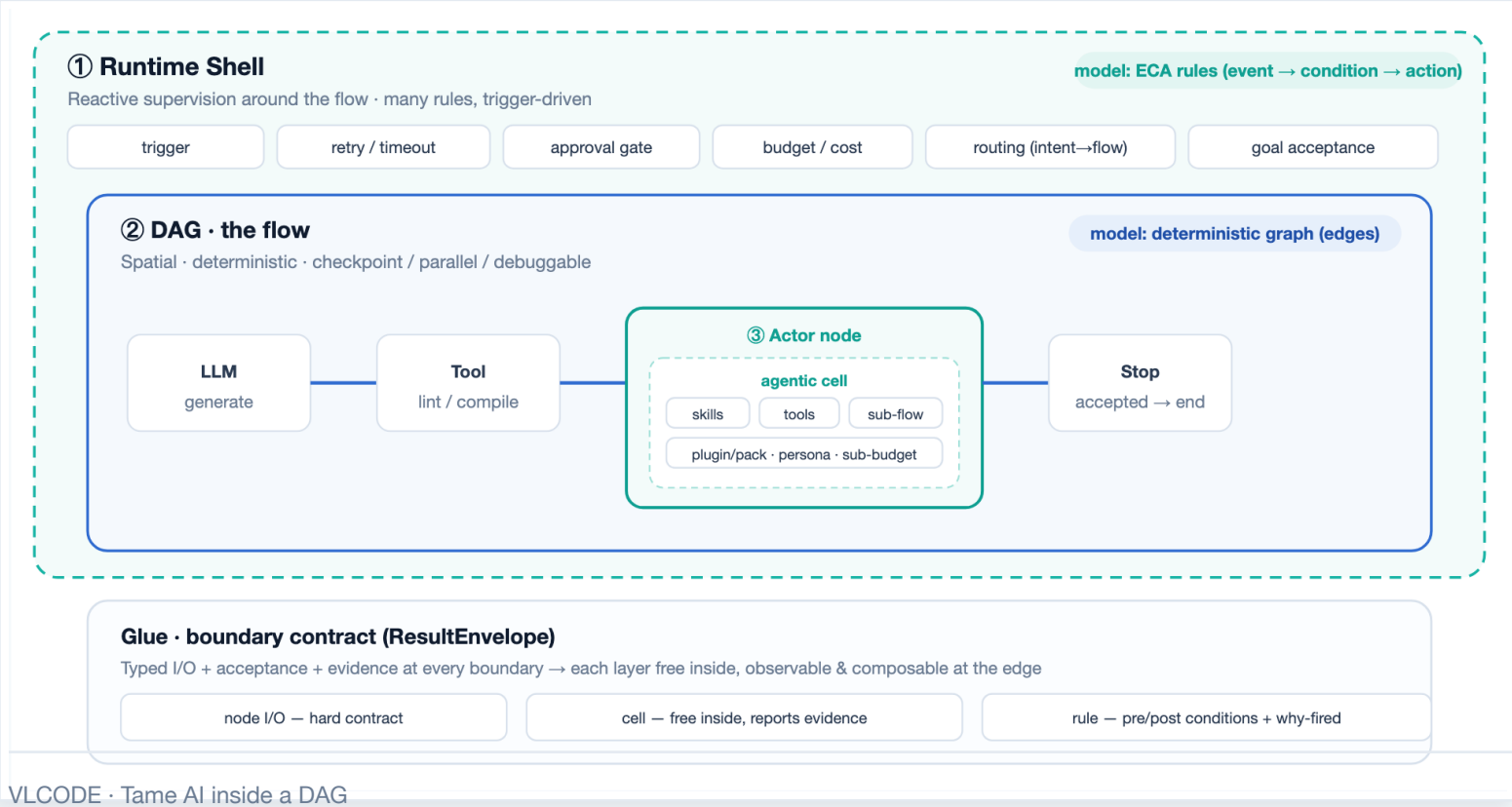
Flow — process DAG



Data — ER model

Three runtimes. One system.

Each layer uses the computational model that fits it — never one nested inside another.



① Shell · rules

Event→condition→action around the flow: trigger, retry, approval, budget, routing, goal acceptance.

② DAG · deterministic

Explicit edges, checkpoint, parallel, node breakpoints. The stable, reproducible spine.

③ Cell · exploratory

An Actor node mounts skills + tools + sub-flows and explores freely — held by a typed contract.

12 DAG VS SKILLS

When to draw a graph. When to hand it to the agent.

Dimension	Skills · agent loop	DAG · workflow
Control flow	Model decides — non-deterministic	Edges & branches fixed — deterministic
Debugging	Hard — no nodes, no breakpoints	Breakpoint / pause / rerun / checkpoint
Parallelism	Mostly sequential	Real Fork / Loop fan-out
Acceptance	Self-declared “done” (soft)	Explicit gates + Score / Review
Observability	Weak	Run ledger + artifact lineage
Authoring cost	Low — a few sentences	Higher — lay out the graph
Best for	Open-ended, judgment, one-off	Repeatable, parallel, audited

Where new logic goes: **repeatable → DAG** · **judgment → skill in an Actor node** · **reactive → shell rule**

THE MOAT

Two products.
One foundation.
One .vflow.

When software scales and AI goes deeper, whoever controls the representation layer controls the ceiling.
And this is just the beginning.

github.com/VisualLogic-AI/VL-Code